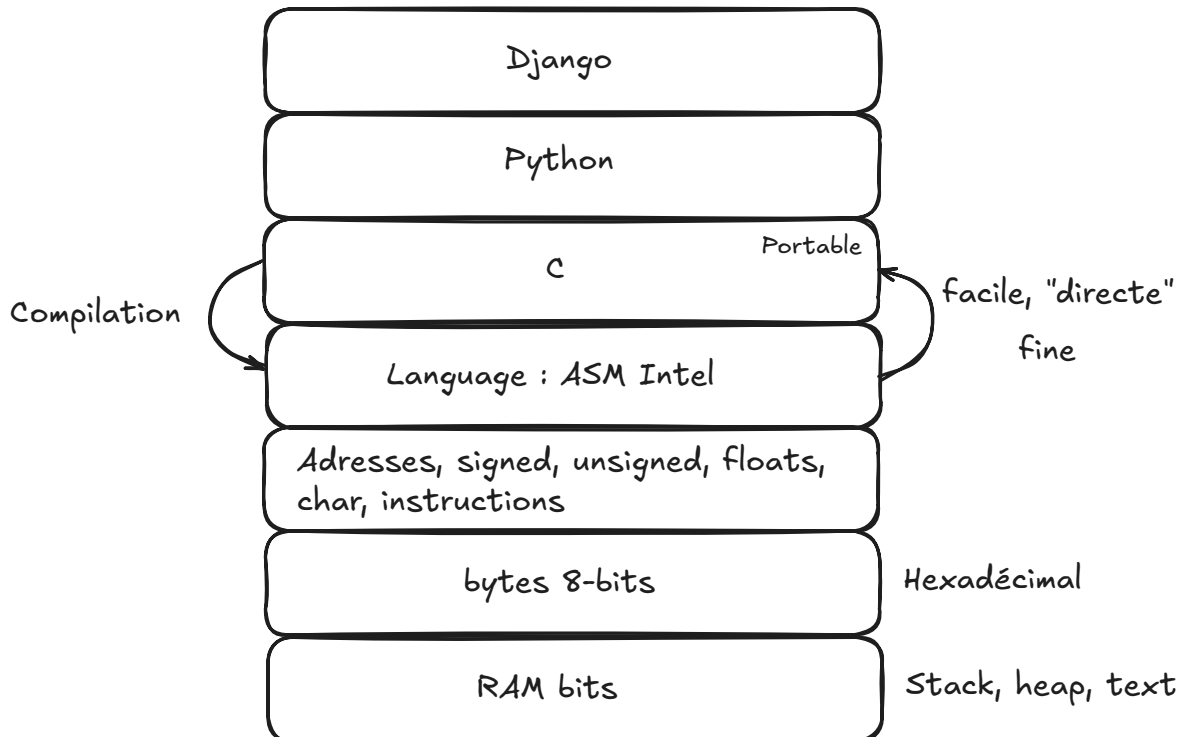


04-03-2025

Dans la RAM, on peut seulement stocké des bits

Quand les octets n'était pas en 8-bits on les appelaient les "mots"



Dans le section "RAM bits" le "text" signifie du code et non du texte

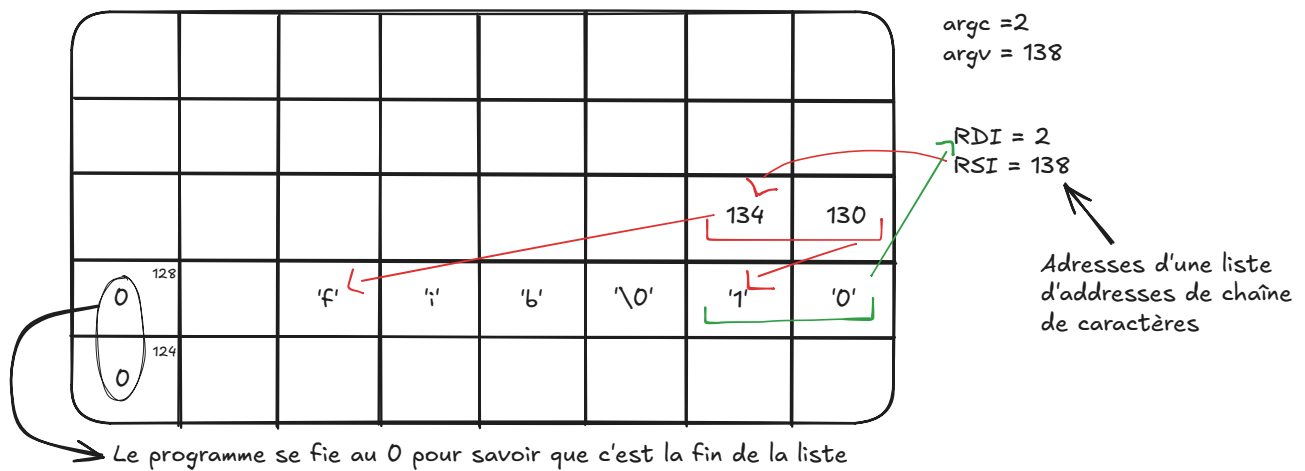
MMU (Memory Management Unit) → partie réécrivant les adresses RAM

On va essayer de reproduire le code en Assembleur d'hier mais en C

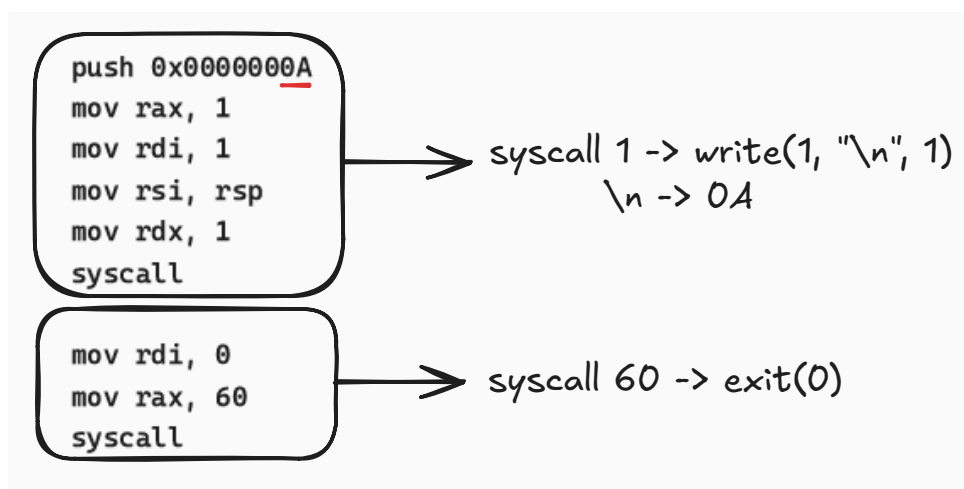
Assembly

```
1 ./fib 10
2 sys.argv = ["fib", "10"]
```

'f' => 0x66 => 01000010



Les cases ont les adresses



[Fabrice Bellard](#) - creator de `ffmpeg`, `tcc` → compilateur C, etc...

C c

```
1 int main(int argc, char **argv) { // "une adresse vers une autre, vers un
  caractere"
2     write(1, *argv, 3);
3     // 1 -> int, *argv -> adresse, 3 -> int
4     exit(0);
5 }
```

char <-> **argv → caractère

char * | *argv → pointeur vers caractère

char ** | argv → adresse de l'adresse de caractère

C c

```
1  int write(int, char*, int);
2  int exit(int);
3
4  int main(int argc, char **argv) {
5      write(1, *argv, 3);
6      exit(0);
7  }
```

`-w` pour ne pas voir les warnings

Cela nous donne './a' (avec un retour à la ligne) car il prends le nom de son fichier et on lui a demandé les trois premiers 3 caractères

`write(1, etc)` → il écrit sur 1 qui est pour les informations

1 → stdout
2 → stderr

En l'exécutant sur Windows PowerShell, le code fonctionne et nous donne `C:\`

`strace` → montre tout les appels systèmes fait par le programme

`hexdump` → affiche en hexa les trucs qu'il a reçu

On veut essayer de faire un `write(1, argv, la_bonne_taille)`

Pour cela, on va scanner la RAM, sachant qu'il connaît le premier caractère jusqu'à ce qu'il trouve 0 pour trouver la taille de la chaîne de caractères

C c

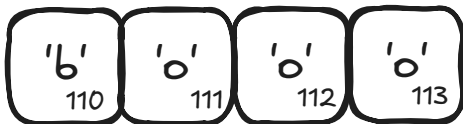
```
1  int write(int, char*, int);
2  int read(int, char*, int);
3  int exit(int);
4
5  int string_size(char *first_char) {
6      int len = 0;
7
8      while (1) { //True
9          if (*first_char == 0) { //ou '\0' pour la fin, octet n'ayant pas le
10                                     //bit à 1 -> comparaison d'un octet à un nb
11              return len;
12          }
13      }
```

```

13         else {
14             len = len + 1;
15             first_char = first_char + 1;
16         }
17     }
18 }
19
20 int main(int argc, char **argv) {
21     write(1, string_size(*argv));
22     exit(0);
23 }

```

ex : first_char = 110



len → 0 1 2 3

length = 3

Les compilateurs d'aujourd'hui de C sont compilés en C.

“Bootstrapper” un langage → compiler un langage avec un autre en premier et après tu recréer un compilateur dans le langage lui-même, le permettant de ce compiler tout seul

On veut éviter d'écrire les trois premières lignes de ce code nommées les **Prototypes de fonctions**

man exit → cela ne me montre pas le bon exit

Ils ont donc des numéros car il y en a plusieurs :

Important

man 1 → commandes

man 2 → syscall

man 3 → fonctions STDI

exit(3) → exit dans la library du C

Cela nous dit que la fonction exit est dans la bibliothèque STDI

#include <stdlib.h> → exit (standard library)

#include <unistd.h> → read, write, ...

Le `#include` cherche le fichier `stdlib.h` et le copie colle (littéralement) dans notre fichier

`gcc -S ex1.c` → pour avoir le fichier en Assembleur
`elf` → format executable

`hexdump -C a.out` → Pour voir le fichier en ASCII
`objdump -Intel -d` → `-d` pour le voir en assembleur,
`-Intel` pour le voir avec la syntaxe de la carte Intel

C c

```
1  int write(int, char*, int);
2  int read(int, char*, int);
3  int exit(int);
4
5  int string_size(char *first_char) {
6      int len = 0;
7
8      while (1) { //True
9          if (*first_char == 0) { //ou '\0' pour la fin, un octet qui n'as
pas le
10                                     //bit à 1 -> comparaison d'un octet à un nb
11              return len;
12          }
13          else {
14              len = len + 1;
15              first_char = first_char + 1;
16          }
17      }
18  }
19
20  int main(int argc, char **argv) {
21      write(1, (*argv+1), string_size>(*argv+1)); //prends l'argument
positionné
22                                     //après l'appel du
fichier
23      exit(0);
24  }
```

						0	0 ¹³⁹
124 ¹³⁸							
		.	/	a	.	o	u
t	\0	t ¹¹⁶	e	s	t	\0	

****argv = ' '**

argv = 138

argc = 124

char **argv

argv +1 = 139

***argv = 125**

```

20
21 int main(int argc, char **argv) {
22     write(1, *argv+1, string_size(*(argv+1)));
23     exit(0);
24 }

```

1 : stdout
2 : stderr

Le but du programme est de calculer la suite de Fibonacci et donc il doit accepter un nombre

Assembly

```

1 int main(int argc, char **argv) {
2     if (argc >= 2) {
3         char n as a char = *(argv + 1);
4     }
5 }
6 exit(0);

```

Pour pouvoir transformer le caractère en chiffre :

'8' - '0' = 8

0x38 - 0x30 = 8

Assembly

```

1 ...
2 int fib(int n) {
3     if (n < 2) {
4         return 1;

```

```

5     }
6     return fib(n-1) + fib(n-2);
7 }
8
9 int main(int argc, char **argv) {
10     if (argc >= 2) {
11         char n_as_a_char = *(argv + 1);
12         int n = n_as_a_char - 0x30;
13         int result = fib(n)
14         char result_as_a_char = 0x30 + result;
15         write(1, &result_as_a_char, 1); //&--- cherche l'adresse du résultat
16     }
17 }
18 exit(0);

```

5 + 0x30 = '5'

'0' + 5 = '5'

Après fib(5) = 8, fib(6) qui doit être 13 ne va pas fonctionner car après 9 en ASCII, on a plus de chiffre et dans ce cas là, cela nous donne un '='.

Il faut que l'on sépare les deux chiffres

Avec 1234 :

Pour obtenir le 4 = 1234 % 10

Pour obtenir le reste = 1234 / 10

Assembly

```

1  int fib(int n){
2      ...
3  }
4
5  void print_int (int value) {
6      int last_digit = value % 10;
7      int remaining_digits = value / 10;
8
9      char last_digit_as_a_char = last_digit + 0x30;
10     write(1, &last_digit_as_a_char, 1);
11     print_int(remaining_digits);
12 }
13
14 int main(int argc, char **argv) {
15     ...
16 }

```

Mais avec ce code là, on print 321 au lieu de 123.

Assembly

```
1 void print_int (int value) {
2     int last_digit = value % 10;
3     int remaining_digits = value / 10;
4
5     char last_digit_as_a_char = last_digit + 0x30;
6     if (remaining_digit) {
7         print_int(remaining_digits);
8     }
9     write(1, &last_digit_as_a_char, 1);
10 }
11
12 int main(int argc, char **argv) {
13     ...
14     char new_line = 0x0A;
15     write(1, &new_line, 1);
16 }
```

Avec ce code là, le code fonctionne seulement jusqu'à Fibonacci de 9

Assembly

```
1 void print_int (int value) {
2     int last_digit = value % 10;
3     int remaining_digits = value / 10;
4
5     char last_digit_as_a_char = last_digit + 0x30;
6     if (remaining_digit) {
7         print_int(remaining_digits);
8     }
9     write(1, &last_digit_as_a_char, 1);
10 }
11
12 int my_atoi(char *number) {
13     int result = 0;
14
15     result = *number - 0x30;
16     return result;
17 }
18
19
20 int main(int argc, char **argv) {
21     if (argc >= 2) {
22         int n = my_atoi(*(argv + 1));
23
24         int result = fib(n);
25         print_int(result);
26 }
```



```
27     char new_line = 0x0A;
28     write(1, &new_line, 1);
29 }
30 exit(0);
31 }
```

Exercice : finir le code

Assembly

```
1  #include <stdlib.h> // exit, ...
2  #include <unistd.h> // read, write, ...
3
4  int string_size(char *first_char) {
5      int len = 0;
6
7      while (1) {
8          if (*first_char == 0) {
9              return len;
10         }
11         else {
12             len = len + 1;
13             first_char = first_char + 1;
14         }
15     }
16 }
17
18 int fib(int n) {
19     if (n < 2) {
20         return 1;
21     }
22     return fib(n-1) + fib(n-2);
23 }
24
25 void print_int(int value) {
26     int last_digit = value % 10;
27     int remaining_digits = value / 10;
28
29     char last_digit_as_a_char = last_digit + 0x30;
30     if (remaining_digits) {
31         print_int(remaining_digits);
32     }
33     write(1, &last_digit_as_a_char, 1);
34 }
35
36
37
```

```

38  int my_atoi(char *number) {
39      int result = 0;
40
41      result = *number - 0x30;
42
43      return result;
44  }
45
46  int main(int argc, char **argv) {
47      if (argc >= 2) {
48          int n = my_atoi(*(argv + 1));
49          int result = fib(n);
50          print_int(result);
51          // write(
52          //      1, // file descriptor, 1 = stdout
53          //      *(argv+1), // address of first char
54          //      string_size(*(argv+1)) // number of chars
55          // );
56          char new_line = 0x0A;
57          write(1, &new_line, 1);
58      }
59      exit(0);
60  }

```

Le problème de ce code est que `my_atoi` prends le chiffre le plus à droite du nombre et fait Fibonacci avec

ex : `fib(56)` → il ne prends en compte que le chiffre le plus à droite et donc 5

Correction de l'exercice

Assembleur

```

1  int my_atoi(char *number) {
2      int result = 0;
3
4      while (*number) {
5          result += *number - 0x30;
6          result *= 10;
7          number++;
8      }
9      result /= 10;
10     return result;
11 }

```